

Motor de Liquidação estilo Pix com Apache Flink (PyFlink)

- Grupo G3 — PDM 2026/1
- Stream processing · Event-time · Exactly-once
- Dataset: PaySim (~6,36M transações, amplificado para $\geq 1\text{GB}$)

Problema

- Pix: dezenas de milhões de transações/dia, liquidação em segundos
- 1) Latência baixíssima -> stream processing
- 2) Correção financeira -> exactly-once (diferença de 1 centavo = falha)
- 3) Resiliência a falhas -> event-time + checkpointing

Arquitetura

- producer -> Kafka tx.raw -> Flink (settlement + AML)
- settlement: ledger keyed por conta, ordem de event-time, RocksDB+MinIO
- sinks exactly-once -> tx.outcomes / aml.alerts
- consumer -> Postgres (saldo final) ; Prometheus+Grafana
- tudo em Docker Compose (roda out-of-the-box)

Exactly-once: a prova

- Baseline batch determinístico = saldo final por conta
- Stream em EXACTLY_ONCE deve bater AO CENTAVO com o baseline
- Demo de falha: matar um TaskManager no meio do processamento
- Recovery do último checkpoint -> reconciliação = ZERO
- AT_LEAST_ONCE + falha -> divergência (double-count)

Detecção de lavagem (AML)

- Ciclo A->B->C->A em janela de 10 min (arestas TRANSFER)
- Structuring: transferências fragmentadas just-below-limit
- Velocidade: muitas transações da mesma conta em janela curta
- Avaliação precision/recall vs rótulo isFraud

Carga com bursts aleatórios

- Picos aleatórios de volume (10–80× a taxa base)
- Injeção de transações sintéticas de ALTO VALOR
- Estressa watermarks, backpressure, checkpoint e recovery
- Modo reproduzível (seed) p/ experimentos; aleatório p/ demo

Experimentos (média $\pm$ desvio, ≥ 3 reps)

- W1 throughput $\times$ paralelismo ; W2 latência P50/95/99
- W3 custo do exactly-once ; W4 tempo de recovery
- W5 reconciliação financeira ; W6 custo de checkpoint
- W7 precision/recall AML ; W8 estado ; W9 bursts

Conclusão

- Demonstra visualmente o valor do exactly-once
- TaskManager morre ao vivo -> recupera -> saldo bate ao centavo
- Stack 100% Docker, reproduzível, em Python (PyFlink)